Paper VIII
Fourth Lean Verification Block

Packet Operator Algorithms, Exact Compilation,
and GPU Realization

Komorebi P. September 6, 2026

Scope and result

This supplement continues the strengthened manuscript dated August 14, 2026. The original manuscript and its source package remain unchanged. All 18 upstream Lean modules and all three earlier mathematical supplements are preserved byte for byte.

Six additional named manuscript results are complete in this release. The cumulative inventory is **46 of 56 manuscript-proved items**, with ten remaining. Four classical inputs are tracked separately. This block adds **50 theorem declarations across five modules**, giving **247 declarations across 23 modules**. Supporting lemmas are included in these counts.

New area	Source items	Declarations
Circulant Fourier diagonalization	4.3	13
Toeplitz circulant embedding	4.4	6
Kronecker and arbitrary tensor contraction	4.5, 4.6	7
Walsh butterfly and exact arithmetic count	4.7	7
Clipped precision allocation	8.5	17
Total added	Six named items	50

All 23 modules passed a fresh cumulative run. Every one of the 247 theorem audits and 22 construction audits used only the allowed foundational axioms or a subset thereof. Both deliberately invalid controls were rejected, and all 17 verifier regression tests passed.

Boundary. The transform identities concern the actual matrices and mathematical algorithms. Arithmetic counts refer to explicit finite event models. These are not GPU timing measurements, CUDA implementation proofs, or an independently formalized fast Fourier transform complexity theorem. The companion Python regression programs are independent mirrors, not executions of the compiled Lean functions.

1. Circulant Fourier diagonalization

Source item 4.3; module CirculantFourier

Let $n > 0$, with coordinates indexed by $\mathbb{Z}/n\mathbb{Z}$. Define the actual circulant by

$$C(c)_{jk} = c_{j-k}.$$

The formal Fourier matrix is

$$F_{jk} = \frac{1}{\sqrt{n}}\,\chi(-jk), \qquad \chi(a) = \exp\!\left(\frac{2\pi i a}{n}\right).$$

The standard character is not left abstract: a checked theorem identifies these entries with

$$\frac{1}{\sqrt{n}}\exp\!\left(-\frac{2\pi i\ \mathrm{val}(j)\,\mathrm{val}(k)}{n}\right).$$

The value representatives enter only this explicit exponential formula. All modular algebra uses the residue classes themselves.

Normalization and inversion. The proof identifies matrix application with the ordinary counting-measure discrete Fourier transform:

$$Fx = n^{-1/2}\,\mathrm{DFT}(x).$$

It separately identifies F^*x with the appropriately scaled inverse transform. Mathlib's checked Fourier inversion theorem and the proved scalar normalization yield both identities

$$\boxed{F^*F = I, \qquad FF^* = I.}$$

No unitarity assumption is inserted.

Convolution and reconstruction. An explicit bijective change of index proves

$$\mathrm{DFT}(C(c)x)_k = \mathrm{DFT}(c)_k\,\mathrm{DFT}(x)_k.$$

This gives the matrix intertwining relation $FC(c) = \mathrm{diag}(\mathrm{DFT}\,c)F$. Multiplication by the proved inverse reconstructs the circulant:

$$\boxed{C(c) = F^*\,\mathrm{diag}(\sqrt{n}\,Fc)F.}$$

The two-transform application formula is also checked for every input vector. The result includes $n = 1$ and arbitrary complex coefficients. Hermiticity is not required.

Claim boundary. This verifies the exact Fourier representation, not an $O(n \log n)$ implementation of the transform. The latter is an additional algorithmic factorization of the same linear map.

2. Toeplitz padding and column-major vectorization

Source items 4.4 and 4.5

Toeplitz embedding. For $n > 0$, define

$$T_{ij} = \begin{cases} c_{i-j}, & j \le i, \\ r_{j-i}, & i < j. \end{cases}$$

The diagonal is supplied by c_0; the coordinate r_0 is unused. The constructed generator of length $2n$ is

$$g = (c_0, \dots, c_{n-1}, 0, r_{n-1}, \dots, r_1),$$

and the padding map is the actual finite-index sum eliminator that appends n zeros.

Lean first proves the upper-left block identity

$$C(g)_{ij} = T_{ij} \qquad (0 \le i, j < n).$$

For $j \le i$, finite modular subtraction has its ordinary value $i - j$. For $i < j$, it has wrapped value $2n + i - j$, whose reversed generator coordinate is $j - i$. Both branches are checked with the actual finite-index arithmetic.

Splitting the product sum into the original and padded halves then gives

$$\boxed{\left(C(g)(x, 0)^T\right)_{0:n} = Tx.}$$

The generator's first half and its middle zero are checked separately. Neither symmetry nor conjugation is involved.

Kronecker application. For $A \in \mathbb{C}^{m\times m}$, $B \in \mathbb{C}^{n\times n}$ and $X \in \mathbb{C}^{n\times m}$, Lean proves

$$\boxed{(A \otimes B)\,\mathrm{vec}(X) = \mathrm{vec}(BXA^T).}$$

The formal vectorization is explicitly column-major: its pair coordinate (j, i) stores X_{ij}. The right factor is the *ordinary transpose*, not the conjugate transpose. This distinction is essential over $\mathbb{C}$.

Finite types of multiplication/accumulation events and stored scalar addresses give

$$\boxed{2n^2m + 2nm^2 \text{ modeled operations}, \qquad n^2 + m^2 \text{ factor entries.}}$$

The explicit Kronecker matrix has n^2m^2 entries. As in the manuscript, the operation model assigns two scalar operations to each multiply-accumulate term.

3. Arbitrary tensor factors

Source item 4.6; module StructuredTensor

Let $A_0, \dots, A_{k-1}$ be square factors of order d. Tensor coordinates are represented by a nested product of k copies of $\{0, \dots, d-1\}$. The empty product is a singleton. Lean proves that the coordinate type has cardinality d^k.

The represented matrix is defined recursively using the actual Kronecker product:

$$M_0 = [1], \qquad M_{k+1} = A_0 \otimes M_k(A_1, \dots, A_k).$$

The algorithm is defined independently. On each slice with fixed first coordinate, it recursively contracts the trailing modes. It then applies A_0 to the resulting first-coordinate fibers:

$$y_{i,u} = \sum_{j=0}^{d-1} (A_0)_{ij} \ \mathrm{Apply}_k(A_1, \dots, A_k; x_{j,\cdot})_u.$$

Induction, the Kronecker entry formula, and finite-sum distributivity prove

$$\boxed{\mathrm{Apply}_k(A; x) = M_k(A)x.}$$

Thus correctness is not inferred merely from agreeing dimension or storage formulas.

Operation events. The event type for $k+1$ modes is a disjoint union of d copies of the trailing event type and the events for the final fiber contraction. Its cardinality obeys

$$W_0 = 0, \qquad W_{k+1} = dW_k + 2d^{k+2}.$$

Lean proves the closed form

$$\boxed{W_k = 2kd^{k+1}.}$$

The factor-address type has cardinality

$$\boxed{kd^2.}$$

These statements include $k = 0$ and $d = 0$. With no factors, the action is the identity on the singleton coordinate type and incurs zero modeled arithmetic. With $d = 0$ and at least one factor, the coordinate type is empty.

Implementation boundary. These are recursive mathematical contractions with exact complex arithmetic. The event model assumes each intermediate slice is computed once and reused. It is not an instrumented execution trace or a cost bound for every evaluator of the raw functional definition. The count excludes indexing, allocation, data movement, fused instruction effects, and the real-operation cost of a complex scalar operation. Those require a separately declared backend model.

4. Walsh butterfly and exact addition count

Source item 4.7; module WalshTransform

The two-point matrix is constructed as

$$H_2 = \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix},$$

and the full family is defined by $H_1 = [1]$ and

$$H_{2N} = H_2 \otimes H_N = \begin{pmatrix} H_N & H_N \\ H_N & -H_N \end{pmatrix}.$$

The two-point orthogonality identity is checked explicitly. Kronecker multiplication and induction then prove

$$\boxed{H_N H_N^T = NI, \qquad N = 2^k.}$$

The ordinary transpose is sufficient because all constructed entries are real, even though the scalar field is $\mathbb{C}$.

Actual butterfly. For the two halves x_0, x_1 of the input, the independently defined procedure returns

$$\mathrm{Walsh}_{2N}(x_0, x_1) = \big(\mathrm{Walsh}_N(x_0 + x_1), \mathrm{Walsh}_N(x_0 - x_1)\big).$$

Lean proves that this vector equals $H_{2N}x$. The argument expands the two first-coordinate cases and applies finite-sum distributivity. It does not assume that the recursive algorithm already represents the matrix.

Exact arithmetic count. Each pair of slices forms one sum and one difference at every trailing coordinate. The event type therefore gives

$$A_0 = 0, \qquad A_{k+1} = 2A_k + 2^{k+1}.$$

The closed form and its logarithmic re-expression are both checked:

$$\boxed{A_k = k2^k = N \log_2 N.}$$

The logarithm in the formal count is the natural-number binary logarithm, with its power-of-two identity proved by Mathlib. The case $N = 1$ has zero arithmetic events.

A further action theorem gives

$$H_N(H_N^T x) = Nx,$$

which identifies the inverse after division by the nonzero scalar N. This result concerns exact arithmetic, not floating-point error accumulation through a butterfly implementation.

5. Clipped precision: existence and multiplier uniqueness

Source item 8.5; module ClippedPrecision

Let E be a finite nonempty set and suppose

$$s_e > 0, \qquad c_e > 0, \qquad m_e > 0, \qquad 0 < \tau \le \sum_e s_e m_e.$$

The feasible set is

$$0 < \varepsilon_e \le m_e, \qquad \sum_e s_e \varepsilon_e = \tau,$$

and the objective is the same logarithmic objective used in item 8.4:

$$J(\varepsilon) = \sum_e c_e \log(1/\varepsilon_e).$$

Strictly positive caps are stated explicitly. A zero or negative cap cannot admit a positive tolerance at that coordinate.

Constructive parameterization. Write $t = 1/\lambda$ and set

$$u_e(t) = \min\{m_e, t c_e / s_e\}, \qquad \Phi(t) = \sum_e s_e u_e(t).$$

Lean proves that Φ is continuous and nondecreasing, with $\Phi(0) = 0$. At the finite positive scale

$$T = \sum_e \frac{m_e s_e}{c_e},$$

every coordinate is saturated, so $u(T) = m$ and $\Phi(T) = \sum_e s_e m_e$. The intermediate value theorem produces $t > 0$ with $\Phi(t) = \tau$. The multiplier is not assumed to exist.

Strictness below saturation. If $t < u$ and $\Phi(t)$ is below the maximum, at least one coordinate is unsaturated at t. That coordinate strictly increases; every other coordinate is nondecreasing. Hence $\Phi(t) < \Phi(u)$. This proves uniqueness of the scale and, by reciprocation, the exact manuscript statement

$$\boxed{\exists!\lambda > 0 : \quad \sum_e s_e \min\left\{m_e, \frac{c_e}{\lambda s_e}\right\} = \tau}$$

when $0 < \tau < \sum_e s_e m_e$.

At the maximal budget, the minimizing vector remains unique but the multiplier need not be. The one-coordinate case $s = c = m = 1$ is checked: both $\lambda = 1$ and $\lambda = 1/2$ produce the same tolerance 1.

6. Clipped precision: global optimality

Source item 8.5, continued

Existence of a multiplier alone does not prove optimality. The formal proof supplies a global supporting inequality, rather than invoking an assumed Karush–Kuhn–Tucker optimality oracle.

Fix the feasible vector $u = u(t)$ obtained above and any feasible competitor ε. For each coordinate, Lean proves

$$\boxed{c_e\left(\frac{\varepsilon_e}{u_e} - 1\right) \le \frac{s_e}{t}(\varepsilon_e - u_e).}$$

When $u_e = tc_e/s_e$, this is equality. When $u_e = m_e$, the capped condition implies $s_e/t \le c_e/m_e$, while $\varepsilon_e - m_e \le 0$. Multiplying by this nonpositive difference gives the required inequality in the correct direction.

The equal-budget constraint then yields

$$\sum_e c_e\left(\frac{\varepsilon_e}{u_e} - 1\right) \le \frac{1}{t}\sum_e s_e(\varepsilon_e - u_e) = 0.$$

Applying the checked inequality $\log z \le z - 1$ for $z > 0$ gives

$$\sum_e c_e \log(\varepsilon_e/u_e) \le 0.$$

Since

$$J(\varepsilon) - J(u) = -\sum_e c_e \log(\varepsilon_e/u_e),$$

we obtain $J(u) \le J(\varepsilon)$ for every feasible competitor.

If $\varepsilon \ne u$, one coordinate has ratio different from 1. The logarithmic inequality is strict there, and its positive weight preserves strictness in the sum. Consequently,

$$\boxed{\varepsilon \ne u \quad \Longrightarrow \quad J(u) < J(\varepsilon).}$$

This proves the unique global minimizing vector for every positive feasible budget, including the maximal endpoint.

Numerical boundary. Existence uses real continuity and the intermediate value theorem. This is a mathematical construction, not an executable floating-point multiplier search with a certified stopping tolerance. The companion exact-rational water-filling routine is an independent regression mirror and is not itself the compiled Lean construction.

7. Verification, regressions, and remaining work

Cumulative release evidence

Lean verification. All 23 modules were rebuilt in a fresh output directory under Lean 4.34.0-rc2 and Mathlib revision

`aeb72f1e9981b6d35b426b16dca7759be3e7ba06`.

Every final module returned exit code zero, with warnings treated as errors. The 247 theorem audits and 22 construction audits used only `propext`, `Classical.choice`, and `Quot.sound`, or subsets thereof. Active proof sources contain no admitted proofs or added axioms. The false-equality and admitted-proof negative controls were rejected. All 17 verifier regression tests passed.

Development failures. Of 20 captured development checks in this block, 12 failed and eight passed. Failures included incomplete development lemmas, import visibility, finite-index instance reduction, and warning diagnostics. Failed source snapshots and compiler logs are retained separately from the successful fresh cumulative run. The first request for interactive runtime recovery was unsupported; noninteractive archive assembly and extraction succeeded, with every part hash and the combined hash verified.

Independent regressions. The Python mirrors passed 180 exact-rational Kronecker cases, 120 tensor cases, 96 Walsh cases, 240 Toeplitz cases, and 600 clipped-allocation cases. The latter include 60 maximal-budget cases and 1,800 rejected infeasible budgets. These are finite checks, not additional universal theorems or executions of the Lean definitions.

A separate complex128 Fourier regression covered 96 cases at lengths 1 through 24. Maximum absolute errors were approximately 3.34×10^{-14} for diagonalization, 2.98×10^{-15} for unitarity, and 2.04×10^{-13} for application, all below the declared 10^{-10} threshold. These numerical checks are not substituted for the symbolic Lean proof.

Ten manuscript-proved items remain:

Item	Remaining result
4.1	Equal block banded count
5.6	Small-core optimal recompression
6.5, 6.6	Triangular level solve and sequential depth
6.7, 6.8	Inverse and Schur support inclusion
7.4, 7.6	Block-Jacobi condition bound and certified preconditioning
9.2	Exact block-discovery certificate
10.3	Acyclic schedule depth

Four classical inputs remain separately recorded. This release does not rerun the original numerical corpus or GPU benchmarks, rebuild Papers I–VII in full, or independently recheck every imported Mathlib proof.